

Hito N°4

Entrega Final

Integrantes: Fernando Palma
Esperanza Cares
Pablo Duarte
Profesor: Eduardo Godoy
Auxiliar: Sebastián M. Castillo
Ayudantes: Tamara Bravo
Joel Riquelme
Nombre de Grupo: Tamal_1

Fecha de realización: 4 de julio de 2025
Fecha de entrega: 4 de julio de 2025
Santiago de Chile

Índice de Contenidos

1. Hito 1	1
1.1. Descripción del conjunto de datos:	1
1.2. Descripción y motivación del problema:	1
1.2.1. Sobre el conjunto de datos	1
1.2.2. Motivación para el Análisis del Conjunto de Datos de Informes Delictivos del Departamento de Policía de Boston	2
1.3. Opción elegida:	2
1.4. Exploración inicial de los datos	3
1.5. Preguntas a resolver mediante el análisis de datos:	4
2. Hito 2	5
2.1. Diagrama Entidad-Relación (E/R)	5
2.2. Traducción al modelo relacional	5
2.2.1. Tabla: Distrito	5
2.2.2. Tabla: Ofensa	5
2.2.3. Tabla: Incidente	5
2.2.4. Tabla: Ocurre_en	6
2.2.5. Tabla: Comete	6
2.3. Análisis de normalización	6
2.3.1. Tabla: Distrito	6
2.3.2. Tabla: Ofensa	6
2.3.3. Tabla: Incidente	7
3. Hito 3	9
3.1. Scripts SQL para implementar la base de datos	9
3.1.1. Script SQL para crear los schemas “Tamal_1” y “Tamal_2”	9
3.1.2. Scripts SQL para crear las tablas del schema “Tamal_1”	9
3.1.3. Scripts SQL para cargar la base de datos a PostgreSQL	12
3.1.4. Scripts SQL para cargar los datos en las tablas	13
3.2. Descripción del estado final de la base de datos con los datos cargados	14
3.3. Consultas a utilizar	16
3.4. Optimización	19
4. Hito 4	21
4.1. Tiempos de Ejecución de Consultas No Optimizadas	21
4.2. Tiempos de Ejecución de Consultas Optimizadas	23
4.3. Comparación de Consultas: “Optimizadas v/s No Optimizadas”	25
4.4. Gráfico de consulta 1	25
4.5. Gráfico de consulta 2	26
4.6. Gráfico de consulta 5	27

Índice de Figuras

Índice de Tablas

Índice de Códigos

1. Hito 1

1.1. Descripción del conjunto de datos:

Enlaces de las fuentes utilizadas. Dataset escogido:

Fuente: *Crimes in Boston*. (2018, 4 de septiembre). Kaggle.

<https://www.kaggle.com/datasets/AnalyzeBoston/crimes-in-boston?select=crime.csv>

Qué tablas o columnas se utilizarán:

Se utilizará la tabla **crime**, pero se normalizará en más tablas para evitar dependencias. Las columnas a utilizar, tomando en cuenta exclusivamente la tabla **crime** son:

1. INCIDENT_NUMBER
2. OFFENSE_CODE
3. OFFENSE_CODE_GROUP
4. OFFENSE_DESCRIPTION
5. DISTRICT
6. REPORTING_AREA
7. OCCURRED_ON_DATE
8. YEAR
9. MONTH
10. DAY_OF_WEEK
11. HOUR
12. UCR_PART
13. STREET

1.2. Descripción y motivación del problema:

1.2.1. Sobre el conjunto de datos

- **Contexto:** Los informes de incidentes delictivos son proporcionados por el Departamento de Policía de Boston (BPD) para documentar los detalles iniciales de un incidente al que responden los oficiales. Este es un conjunto de datos que contiene registros del nuevo sistema de informes de delitos, el cual incluye un conjunto reducido de campos centrados en capturar el tipo de incidente, así como cuándo y dónde ocurrió.

- **Contenido:** Los registros comienzan el 14 de junio de 2015 y continúan hasta el 3 de septiembre de 2018.
- **Fuente original:** Los datos son proporcionados por Analyze Boston. La versión más actualizada puede encontrarse en el siguiente enlace.

Fuente: *Crime incidents Reports*. (August 2015 - to date) (Source: New System) - Analyze Boston. (s. f.).

<https://data.boston.gov/dataset/crime-incident-reports-august-2015-to-date-source-new-system>

1.2.2. Motivación para el Análisis del Conjunto de Datos de Informes Delictivos del Departamento de Policía de Boston

Este dataset de informes delictivos del Departamento de Policía de Boston (2015-2018) es ideal para:

1. Impacto real:

- Identificar patrones de crimen (horarios, zonas críticas).
- Apoyar decisiones policiales y políticas públicas.

2. Datos valiosos:

- Clasificación por tipo de delito (UCR), ubicación y fecha.
- Perfecto para mapas de calor, series de tiempo y modelos predictivos.

3. Habilidades demostradas:

- Limpieza de datos, visualización (Confección de página web) y análisis geoespacial.
- Posibilidad de usar Python/R o herramientas como SQL.

Enfoque conciso:

"Transformar datos crudos en insights accionables para mejorar la seguridad urbana."

1.3. Opción elegida:

Se realizará un reporte de **Análisis de datos**, formulando **6 preguntas de investigación** (tendencias, predicciones, etc.).

1.4. Exploración inicial de los datos

- **Total de valores:** 4147949 (13C x 319073F).
- **Cantidad de filas por tabla:** Tabla crime (cantidad de filas, 319073).
- **Valores nulos:** 65952
- **Porcentaje de valores nulos:** 1.589991 %
- **Distribución o valores típicos de atributos relevantes:**
 1. OFFENSE_CODE_GROUP (Grupos de delitos principales)
 - Motor Vehicle Accident: 12 %
 - Larceny: 8 %
 2. DISTRICT (Distritos con más incidentes)
 - B2: 16 %
 - C11: 13 %
 3. OFFENSE_DESCRIPTION (Descripciones específicas)
 - SICK/INJURED/MEDICAL: 6 % del total.
 - INVESTIGATE PERSON: 6 % del total
- **Columna que admite valores múltiples:** La columna **OCCURRED_ON_DATE** admite fecha y hora en la misma columna.
- **Tipo y dominio de cada atributo:**

Tabla 1: Descripción de Atributos del Dataset de Crímenes en Boston

Nombre del Atributo	Tipo de Dato (PSQL)	Admite Múltiples Valores
INCIDENT_NUMBER	VARCHAR(10)	No
OFFENSE_CODE	VARCHAR(5)	No
OFFENSE_CODE_GROUP	TEXT	No
OFFENSE_DESCRIPTION	TEXT	No
DISTRICT	VARCHAR(4)	No
REPORTING_AREA	SMALLINT	No
OCCURRED_ON_DATE	TIMESTAMP	Sí
YEAR	SMALLINT	No
MONTH	SMALLINT	No
DAY_OF_WEEK	VARCHAR(9)	No
HOURL	SMALLINT	No
UCR_PART	VARCHAR(15)	No
STREET	TEXT	No

1.5. Preguntas a resolver mediante el análisis de datos:

- ¿Cómo ha evolucionado la cantidad de crímenes reportados por distrito a lo largo de los años?
- ¿Cuáles son los tipos de crimen más frecuentes por día de la semana y cómo varían?
- ¿En qué horas del día ocurren más delitos y cómo cambia este patrón según el tipo de crimen?
- ¿Existen diferencias significativas en la distribución de delitos entre meses o estaciones del año?
- ¿Qué calles concentran mayor cantidad de crímenes y que tipo de delitos predominan en ellas?
- ¿Hay alguna relación entre el tipo de crimen y el distrito de donde ocurre?

2. Hito 2

2.1. Diagrama Entidad-Relación (E/R)

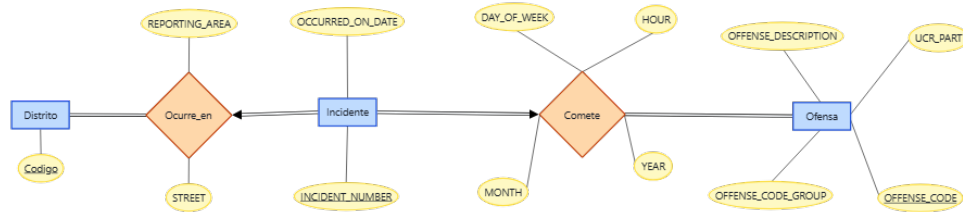


Figura 1: Diagrama Entidad-Relación.

Originalmente el dataset contenía solo una tabla masiva con 13 columnas, para el diagrama se tomó la decisión de dividirlo en 3 entidades con 2 relaciones. La lógica de separa las ofensas es debido a que tienen un número único de posibilidades mucho menos al número de incidentes, lo mismo se da para los distritos.

2.2. Traducción al modelo relacional

2.2.1. Tabla: Distrito

Atributo	Dominio	Clave
Código	VARCHAR(4)	PK

2.2.2. Tabla: Ofensa

Atributo	Dominio	Clave
OFFENSE_CODE	VARCHAR(5)	PK
OFFENSE_CODE_GROUP	TEXT	
OFFENSE_DESCRIPTION	TEXT	
UCR_PART	VARCHAR(15)	

2.2.3. Tabla: Incidente

Atributo	Dominio	Clave
INCIDENT_NUMBER	VARCHAR(10)	PK
OCCURRED_ON_DATE	TIMESTAMP	

2.2.4. Tabla: Ocorre_en

Atributo	Dominio	Clave
INCIDENT_NUMBER	VARCHAR(10)	PK, FK
Código	VARCHAR(4)	FK
REPORTING_AREA	SMALLINT	
STREET	TEXT	

2.2.5. Tabla: Comete

Atributo	Dominio	Clave
INCIDENT_NUMBER	VARCHAR(10)	PK, FK
OFFENSE_CODE	VARCHAR(5)	PK, FK
DAY_OF_WEEK	VARCHAR(9)	
HOURL	SMALLINT	
MONTH	SMALLINT	
YEAR	SMALLINT	

2.3. Análisis de normalización

2.3.1. Tabla: Distrito

La tabla **Distrito** se define con los siguientes atributos:

- **Codigo** (PK)

Análisis de Forma Normal:

- **1NF**: La tabla cumple con la Primera Forma Normal, ya que todos sus atributos son atómicos y posee una clave primaria (**Codigo**).
- **2NF**: Dado que la clave primaria es simple, no pueden existir dependencias parciales. Por lo tanto, la tabla está en Segunda Forma Normal.
- **3NF**: No existen atributos no clave, por lo que no pueden existir dependencias transitivas. Por lo tanto, la tabla está en Tercera Forma Normal.
- **BCNF**: Para toda dependencia funcional no trivial $X \rightarrow Y$, X debe ser una superclave. En esta tabla, la única dependencia es **Codigo** $\rightarrow$ **Codigo** (trivial) o no hay otros atributos que dependan de **Codigo**. Por lo tanto, la tabla **Distrito** se encuentra en BCNF.

2.3.2. Tabla: Ofensa

La tabla **Ofensa** se define con los siguientes atributos:

- OFFENSE_CODE (PK)
- OFFENSE_CODE_GROUP
- OFFENSE_DESCRIPTION
- UCR_PART

Las dependencias funcionales identificadas son:

- OFFENSE_CODE $\rightarrow$ OFFENSE_CODE_GROUP
- OFFENSE_CODE $\rightarrow$ OFFENSE_DESCRIPTION
- OFFENSE_CODE $\rightarrow$ UCR_PART

Análisis de Forma Normal:

- **1NF**: La tabla cumple con la Primera Forma Normal, ya que todos sus atributos son atómicos y posee una clave primaria (OFFENSE_CODE).
- **2NF**: La clave primaria es simple, por lo que no hay dependencias parciales. La tabla está en Segunda Forma Normal.
- **3NF**: Todos los atributos no clave (OFFENSE_CODE_GROUP, OFFENSE_DESCRIPTION, UCR_PART) dependen directamente de la clave primaria OFFENSE_CODE. No existen dependencias transitivas entre atributos no clave. Por lo tanto, la tabla está en Tercera Forma Normal.
- **BCNF**: El único determinante de las dependencias funcionales no triviales es OFFENSE_CODE, que es la clave primaria (y por ende una superclave). Por lo tanto, la tabla Ofensa se encuentra en BCNF.

2.3.3. Tabla: Incidente

La tabla Incidentes se define con los siguientes atributos:

- INCIDENT_NUMBER (PK)
- OCURRED_ON_DATE

La dependencia funcional identificada es:

- INCIDENT_NUMBER $\rightarrow$ OCURRED_ON_DATE

Análisis de Forma Normal:

- **1NF**: La tabla cumple con la Primera Forma Normal, ya que todos sus atributos son atómicos y posee una clave primaria (INCIDENT_NUMBER).

- **2NF:** La tabla cumple 1NF. La clave primaria es simple (compuesta por un solo atributo: INCIDENT_NUMBER). El único atributo no clave (OCURRED_ON_DATE) depende completamente de INCIDENT_NUMBER. La tabla cumple con la Segunda Forma Normal.
- **3NF:** La tabla cumple con 2NF. No hay otros atributos no clave adicionales. No existen dependencias transitivas, ya que todos los atributos no clave (en este caso solo uno) dependen directamente de la clave primaria. La tabla cumple con la Tercera Forma Normal.
- **BCNF:** La tabla cumple con 1NF. La única dependencia funcional en la tabla es INCIDENT_NUMBER $\rightarrow$ OCURRED_ON_DATE. INCIDENT_NUMBER es la clave primaria, por lo tanto también es una superclave. La tabla cumple con la Forma Normal de Boyce-Codd.

3. Hito 3

3.1. Scripts SQL para implementar la base de datos

3.1.1. Script SQL para crear los schemas “Tamal_1” y “Tamal_2”

Código 1: SCHEMAS “Tamal_1” y “Tamal_2”

```
1 CREATE SCHEMA Tamal_1;  
2 CREATE SCHEMA Tamal_2;
```

Se crean dos esquemas distintos en PostgreSQL:

- **Tamal_1**: Contendrá las tablas normalizadas, separadas por entidad.
- **Tamal_2**: Funcionará como zona de staging o carga temporal, con una única tabla que contiene todos los datos crudos directamente del archivo CSV.

3.1.2. Scripts SQL para crear las tablas del schema “Tamal_1”

Código 2: Script SQL para crear la tabla “Distrito”

```
1 CREATE TABLE Tamal_1.Distrito (  
2     Codigo VARCHAR(4) NOT NULL,  
3     PRIMARY KEY (Codigo)  
4 );
```

Esta tabla contiene los distintos distritos policiales donde se registran crímenes en la ciudad de Boston. El atributo **Codigo** representa un identificador único para cada distrito.

- **Codigo**: Clave primaria. Identificador del distrito (VARCHAR(4)).

Es una tabla de tipo **dimensión** utilizada para referenciar la ubicación administrativa del incidente.

Código 3: Script SQL para crear la tabla “Ofensa”

```
1 CREATE TABLE Tamal_1.Ofensa (  
2     OFFENSE_CODE VARCHAR(5) NOT NULL,  
3     OFFENSE_CODE_GROUP TEXT NOT NULL,  
4     OFFENSE_DESCRIPTION TEXT NOT NULL,  
5     UCR_PART VARCHAR(15) NOT NULL,  
6     PRIMARY KEY (OFFENSE_CODE)  
7 );
```

Almacena información relacionada con los distintos tipos de delitos registrados. Cada fila representa una categoría específica de crimen.

- **OFFENSE_CODE**: Clave primaria. Código del delito (VARCHAR(5)).
- **OFFENSE_CODE_GROUP**: Grupo al que pertenece el delito (TEXT).
- **OFFENSE_DESCRIPTION**: Descripción del delito (TEXT).
- **UCR_PART**: Clasificación según el sistema UCR (VARCHAR(15)).

Se trata de una tabla **dimensión** que permite categorizar los incidentes delictivos.

Código 4: Script SQL para crear la tabla “Incidente”

```
1 CREATE TABLE Tamal_1.Incidente (  
2     INCIDENT_NUMBER VARCHAR(10) NOT NULL,  
3     OCURRED_ON_DATE TIMESTAMP NOT NULL,  
4     PRIMARY KEY (INCIDENT_NUMBER)  
5 );
```

Representa los distintos incidentes delictivos reportados. Cada incidente tiene una fecha y hora de ocurrencia.

- **INCIDENT_NUMBER**: Clave primaria. Identificador único del incidente (VARCHAR(10)).
- **OCURRED_ON_DATE**: Fecha y hora en que ocurrió el incidente (TIMESTAMP).

Es una tabla **central o de hechos**, ya que se relaciona con otras entidades y contiene la ocurrencia principal del delito.

Código 5: Script SQL para crear la tabla “Ocurre_en”

```
1 CREATE TABLE Tamal_1.Ocurre_en (  
2     INCIDENT_NUMBER VARCHAR(10) NOT NULL,  
3     Codigo VARCHAR(4) NOT NULL,  
4     REPORTING_AREA SMALLINT NOT NULL,  
5     STREET TEXT NOT NULL,  
6     PRIMARY KEY (INCIDENT_NUMBER),  
7     FOREIGN KEY (INCIDENT_NUMBER) REFERENCES Tamal_1.Incidente  
8         ↪ (INCIDENT_NUMBER)  
9         ON UPDATE CASCADE  
10        ON DELETE CASCADE,  
11     FOREIGN KEY (Codigo) REFERENCES Tamal_1.Distrito (Codigo)  
12        ON UPDATE CASCADE  
13        ON DELETE RESTRICT  
14 );
```

Relaciona un incidente con el lugar físico donde ocurrió. Además del distrito, considera una zona de reporte y la calle específica.

- **INCIDENT_NUMBER**: Clave foránea. Referencia al incidente ocurrido.

- **Codigo:** Clave foránea. Código del distrito donde ocurrió.
- **REPORTING_AREA:** Subzona dentro del distrito (SMALLINT).
- **STREET:** Calle específica del incidente (TEXT).

Es una tabla de relación entre las entidades **Incidente** y **Distrito**.

Código 6: Script SQL para crear la tabla “Comete”

```
1 CREATE TABLE Tamal_1.Comete (  
2     INCIDENT_NUMBER VARCHAR(10) NOT NULL,  
3     OFFENSE_CODE VARCHAR(5) NOT NULL,  
4     DAY_OF_WEEK VARCHAR(9) NOT NULL CHECK (DAY_OF_WEEK IN  
5         ⇨ ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')),  
6     HOUR SMALLINT NOT NULL CHECK (HOUR >= 0 AND HOUR <= 23),  
7     MONTH SMALLINT NOT NULL CHECK (MONTH >= 1 AND MONTH <= 12),  
8     YEAR SMALLINT NOT NULL,  
9     PRIMARY KEY (INCIDENT_NUMBER, OFFENSE_CODE),  
10    FOREIGN KEY (INCIDENT_NUMBER) REFERENCES Tamal_1.Incidente  
11    ⇨ (INCIDENT_NUMBER)  
12    ON UPDATE CASCADE  
13    ON DELETE CASCADE,  
14    FOREIGN KEY (OFFENSE_CODE) REFERENCES Tamal_1.Ofensa  
15    ⇨ (OFFENSE_CODE)  
16    ON UPDATE CASCADE  
17    ON DELETE RESTRICT  
18 );
```

Relaciona cada incidente con una ofensa específica cometida, junto con información temporal adicional. Se trata de una relación de muchos a muchos entre las tablas **Incidente** y **Ofensa**.

- **INCIDENT_NUMBER:** Clave foránea. Referencia al incidente reportado.
- **OFFENSE_CODE:** Clave foránea. Tipo de delito cometido.
- **DAY_OF_WEEK:** Día de la semana en que ocurrió (VARCHAR(9)).
- **HOUR:** Hora del día (SMALLINT, entre 0 y 23).
- **MONTH:** Mes del año (SMALLINT, entre 1 y 12).
- **YEAR:** Año del incidente (SMALLINT).

Esta tabla permite realizar análisis temporales y por tipo de delito de manera más eficiente.

3.1.3. Scripts SQL para cargar la base de datos a PostgreSQL

Código 7: Script SQL para crear la tabla general “Tamal_2.Crimenes”

```

1 CREATE TABLE Tamal_2.Crimenes (
2   INCIDENT_NUMBER VARCHAR(10) NOT NULL,
3   OFFENSE_CODE VARCHAR(5) NOT NULL,
4   OFFENSE_CODE_GROUP TEXT NOT NULL,
5   OFFENSE_DESCRIPTION TEXT NOT NULL,
6   Codigo VARCHAR(4) NOT NULL,
7   REPORTING_AREA SMALLINT NOT NULL,
8   OCURRED_ON_DATE TIMESTAMP NOT NULL,
9   YEAR SMALLINT NOT NULL,
10  MONTH SMALLINT NOT NULL CHECK (MONTH BETWEEN 1 AND 12),
11  DAY_OF_WEEK VARCHAR(9) NOT NULL CHECK (DAY_OF_WEEK IN
    ↳ ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')),
12  HOUR SMALLINT NOT NULL CHECK (HOUR BETWEEN 0 AND 23),
13  UCR_PART VARCHAR(15) NOT NULL,
14  STREET TEXT NOT NULL
15 );

```

Se crea una tabla temporal en el esquema Tamal_2 que replica exactamente la estructura del archivo .csv original. Esta tabla contiene los datos sin normalizar y será la fuente de datos para poblar las tablas en Tamal_1.

Código 8: Script para subir el archivo csv “crimenes.csv” al entorno de PostgreSQL

```

1 scp -P 325 "C:\Users\pablo\OneDrive\Escritorio\Universidad Bruh\7mo Semestre\Bases de
    ↳ Datos\Proyecto de Base de Datos\crimenes.csv"
    ↳ cc3201@cc3201.dcc.uchile.cl:/home/cc3201/

```

Este comando utiliza scp (secure copy) para copiar el archivo CSV desde el computador local al servidor remoto de la Universidad. El puerto 325 se especifica para conectar correctamente.

Código 9: Script para copiar el archivo csv “crimenes.csv” a la tabla general “Tamal_2.Crimenes”

```

1 \COPY Tamal_2.Crimenes FROM '/home/cc3201/crimenes.csv' WITH (FORMAT csv,
    ↳ HEADER true, DELIMITER ';');

```

Este comando se ejecuta desde la terminal psql y carga los datos desde el archivo crimenes.csv a la tabla Tamal_2.Crimenes. Se utiliza:

- **HEADER true** para indicar que la primera fila contiene los nombres de las columnas.
- **DELIMITER ';' ya que el archivo está separado por punto y coma.**

3.1.4. Scripts SQL para cargar los datos en las tablas

Código 10: Script SQL para copiar los datos correspondientes de la tabla general “Tamal_2.Crimenes” a la tabla “Tamal_1.Distrito”

```
1 INSERT INTO Tamal_1.Distrito (Codigo)
2 SELECT DISTINCT Codigo
3 FROM Tamal_2.Crimenes;
```

Se extraen los códigos únicos de distrito desde los datos crudos y se insertan en la tabla Distrito, eliminando duplicados con DISTINCT.

Código 11: Script SQL para copiar los datos correspondientes de la tabla general “Tamal_2.Crimenes” a la tabla “Tamal_1.Ofensa”

```
1 INSERT INTO Tamal_1.Ofensa (OFFENSE_CODE, OFFENSE_CODE_GROUP,
   ↪ OFFENSE_DESCRIPTION, UCR_PART)
2 SELECT DISTINCT OFFENSE_CODE, OFFENSE_CODE_GROUP,
   ↪ OFFENSE_DESCRIPTION, UCR_PART
3 FROM Tamal_2.Crimenes;
```

Se pobla la tabla Ofensa con los códigos únicos de delitos y sus respectivas clasificaciones y descripciones.

Código 12: Script SQL para copiar los datos correspondientes de la tabla general “Tamal_2.Crimenes” a la tabla “Tamal_1.Incidente”

```
1 INSERT INTO Tamal_1.Incidente (INCIDENT_NUMBER, OCURRED_ON_DATE)
2 SELECT DISTINCT INCIDENT_NUMBER, OCURRED_ON_DATE
3 FROM Tamal_2.Crimenes;
```

Carga la información de incidentes únicos y sus fechas a la tabla Incidente.

Código 13: Script SQL para copiar los datos correspondientes de la tabla general “Tamal_2.Crimenes” a la tabla “Tamal_1.Ocurre_en”

```
1 INSERT INTO Tamal_1.Ocurre_en (INCIDENT_NUMBER, Codigo,
   ↪ REPORTING_AREA, STREET)
2 SELECT DISTINCT INCIDENT_NUMBER, Codigo, REPORTING_AREA, STREET
3 FROM Tamal_2.Crimenes;
```

Asocia los incidentes con su ubicación específica (distrito, subzona y calle).

Código 14: Script SQL para copiar los datos correspondientes de la tabla general “Tamal_2.Crimenes” a la tabla “Tamal_1.Comete”

```
1 INSERT INTO Tamal_1.Comete (INCIDENT_NUMBER, OFFENSE_CODE,
   ↪ DAY_OF_WEEK, HOUR, MONTH, YEAR)
```



```
2 SELECT DISTINCT INCIDENT_NUMBER, OFFENSE_CODE, DAY_OF_WEEK,  
   ↪ HOUR, MONTH, YEAR  
3 FROM Tamal_2.Crimenes;
```

Relaciona cada incidente con la ofensa cometida, incluyendo información temporal como el día, hora y año del suceso. Esta tabla permite realizar análisis por tipo de delito y temporalidad.

3.2. Descripción del estado final de la base de datos con los datos cargados

Cantidad de filas por tabla

Tabla 2: Cantidad de filas por tabla cargada

Tabla	Filas cargadas (estimado)
Tamal_1.Distrito	12
Tamal_1.Ofensa	217
Tamal_1.Incidente	264.126
Tamal_1.Ocurre_en	264126
Tamal_1.Comete	297713

Problemas detectados y procesamiento necesario

- El conjunto de datos original no estaba en formato .csv, sino en texto simple, por lo que tuvo que ser convertido utilizando herramientas externas.
- Se eliminaron algunos atributos del conjunto de datos original antes de subirlo al servidor, por lo que no existen comandos de psql para eliminarlos.
- Mediante el uso de **DISTINCT**, se eliminaron las tuplas duplicadas y, al restringir las tablas con **NOT NULL**, se omitieron los datos nulos que se indicaron en el Hito 1.

Capturas de pantalla

```
cc3201=# SELECT COUNT(*) AS total_rows
FROM comete;
total_rows
-----
297713
(1 row)
```

```
cc3201=# select * from tamal_1.comete limit 10;
incident_number | offense_code | day_of_week | hour | month | year
-----
I182055753 | 3006 | Sunday | 16 | 7 | 2018
I182025362 | 2405 | Thursday | 13 | 4 | 2018
I182054577 | 1402 | Wednesday | 16 | 7 | 2018
I182058209 | 2907 | Tuesday | 2 | 7 | 2018
I182065593 | 3802 | Friday | 8 | 8 | 2018
I182058087 | 3208 | Saturday | 13 | 7 | 2018
I182011280 | 1848 | Monday | 9 | 2 | 2018
I182042451 | 521 | Saturday | 2 | 6 | 2018
I182052609 | 802 | Wednesday | 22 | 7 | 2018
I182011840 | 617 | Wednesday | 12 | 2 | 2018
(10 rows)
```

```
cc3201=# select * from tamal_1.incidente limit 10;
incident_number | occurred_on_date
-----
I182007266 | 2018-01-28 01:24:00
I182038958 | 2018-03-08 00:00:00
I182065517 | 2018-08-16 19:32:00
I182021261 | 2018-03-21 15:49:00
I182055141 | 2018-07-13 13:23:00
I182009893 | 2018-02-06 19:56:00
I182004457 | 2018-01-17 12:02:00
I182040384 | 2018-05-26 00:00:00
I182045519 | 2018-06-12 11:06:00
I182005886 | 2018-01-23 06:12:00
(10 rows)
```

```
cc3201=# select * from tamal_1.ofensa limit 10;
offense_code | offense_code_group | offense_description | ucr_part
-----
1848 | Drug Violation | DRUGS - POSS CLASS B - INTENT TO MFR DIST DISP | Part Two
3123 | Explosives | EXPLOSIVES - TURNED IN OR FOUND | Part Three
403 | Aggravated Assault | ASSAULT & BATTERY D/W - OTHER | Part One
2663 | Other | VIOLATION - CITY ORDINANCE CONSTRUCTION PERMIT | Part Two
541 | Commercial Burglary | BURGLARY - COMMERCIAL - ATTEMPT | Part One
2995 | Violations | VAL - OPERATING WITHOUT LICENSE | Part Two
2511 | Other | KIDNAPPING - ENTICING OR ATTEMPTED | Part Two
2007 | Restraining Order Violations | VIOL. OF RESTRAINING ORDER W NO ARREST | Part Two
521 | Residential Burglary | BURGLARY - RESIDENTIAL - ATTEMPT | Part One
2403 | Disorderly Conduct | DISTURBING THE PEACE | Part Two
(10 rows)
```

Figura 2: Registro de las tablas COMETE, INCIDENTE y OFENSA.

```
cc3201=# select * from tamal_1.districto limit 10;
codigo
-----
E13
B3
D4
A7
E18
E5
B2
C6
D14
A15
(10 rows)
```

```
cc3201=# select * from tamal_1.ocurre_en limit 10;
incident_number | codigo | reporting_area | street
-----
I172107418 | E13 | 506 | BOURNE ST
I182038009 | D4 | 619 | WESTLAND AVE
I182042563 | C6 | 175 | SOUTHAMPTON ST
I182016876 | C11 | 918 | SUMNER ST
I182031212 | C11 | 352 | CHARLES ST
I182044699 | A1 | 111 | WASHINGTON ST
I182005139 | D14 | 796 | COMMONWEALTH AVE
I182032040 | A15 | 45 | MEDFORD ST
I182057195 | E18 | 494 | RIVER ST
I182048582 | D4 | 628 | PARK DR
(10 rows)
```

Figura 3: Registro de las tablas DISTRITO y OCURRE.

3.3. Consultas a utilizar

Basadas en las preguntas a resolver del Hito 1:

Consulta 1: Evolución anual de la cantidad de crímenes por distrito

Se genera una tabla con los distritos como filas y los años como columnas para visualizar el cambio en el período de tiempo del conjunto de datos.

Código 15: Consulta 1.

```
1 SELECT
2     Codigo AS Distrito,
3     COUNT(*) FILTER (WHERE YEAR = 2015) AS "2015",
4     COUNT(*) FILTER (WHERE YEAR = 2016) AS "2016",
5     COUNT(*) FILTER (WHERE YEAR = 2017) AS "2017",
6     COUNT(*) FILTER (WHERE YEAR = 2018) AS "2018"
7 FROM Tamal_1.Comete C
8 JOIN Tamal_1.Ocorre_en O USING (INCIDENT_NUMBER)
9 GROUP BY Codigo
10 ORDER BY Codigo;
```

Consulta 2: Crímenes más frecuentes por día de la semana

Se seleccionan las tres descripciones de crímenes más comunes por día de la semana para observar las diferencias.

Código 16: Consulta 2.

```
1 SELECT *
2 FROM (
3     SELECT
4         DAY_OF_WEEK,
5         O2.OFFENSE_DESCRIPTION,
6         COUNT(*) AS Total,
7         RANK() OVER (
8             PARTITION BY DAY_OF_WEEK
9             ORDER BY COUNT(*) DESC
10        ) AS Rank
11 FROM Tamal_1.Comete C
12 JOIN Tamal_1.Ofensa O2 ON C.OFFENSE_CODE = O2.OFFENSE_CODE
13 GROUP BY DAY_OF_WEEK, O2.OFFENSE_DESCRIPTION
14 ) AS ranked
15 WHERE Rank <= 3;
```

Consulta 3: Hora del día que ocurren más delitos según su tipo.

Se agrupan los incidentes cometidos por hora y se extrae la hora con mayor frecuencia para cada descripción de ofensa. Con esto, se pueden analizar los horarios comunes de las ofensas.

Código 17: Consulta 3.

```
1 SELECT
2   o.OFFENSE_DESCRIPTION,
3   com.HOUR AS HoraMasComun
4 FROM Tamal_1.Ofensa o
5 JOIN (
6   SELECT
7     OFFENSE_CODE,
8     HOUR,
9     COUNT(*) AS Frecuencia,
10    ROW_NUMBER() OVER (PARTITION BY OFFENSE_CODE ORDER BY
11      ↪ COUNT(*) DESC) AS rn
12 FROM Tamal_1.Comete
13 GROUP BY
14   OFFENSE_CODE,
15   HOUR
16 ) AS com ON o.OFFENSE_CODE = com.OFFENSE_CODE
17 WHERE
18   com.rn = 1
19 ORDER BY
20   o.OFFENSE_DESCRIPTION;
```

Consulta 4: Diferencia mensual en la distribución de delitos.

Código 18: Consulta 4.

```
1 WITH recuento AS (
2   SELECT
3     o.codigo AS distrito,
4     DATE_TRUNC('month', i.occurred_on_date) AS mes,
5     COUNT(*) AS total_incidentes
6 FROM tamal_1.incidente i
7 JOIN tamal_1.ocurre_en o USING (incident_number)
8 WHERE EXTRACT(YEAR FROM i.occurred_on_date)
9   BETWEEN 2015 AND 2025
10 GROUP BY 1,2
11 ),
12 con_diferencia AS (
13   SELECT
14     distrito,
```

```
15     mes,
16     total_incidentes,
17     LAG(total_incidentes)
18     OVER (PARTITION BY distrito ORDER BY mes) AS total_mes_anterior
19 FROM recuento
20 )
21 SELECT
22     distrito,
23     mes,
24     total_incidentes,
25     COALESCE(total_mes_anterior,0) AS total_mes_prev,
26     ROUND(
27         100.0 * (total_incidentes - total_mes_anterior)
28         / NULLIF(total_mes_anterior,0), 2) AS pct_cambio
29 FROM con_diferencia
30 ORDER BY distrito, mes;
```

Consulta 5: Calles con mayor cantidad de delitos y sus tipos más comunes.

Código 19: Consulta 5.

```
1
2 WITH ranking_calles AS (
3     SELECT
4         street,
5         COUNT(*) AS total_incidentes,
6         RANK() OVER (ORDER BY COUNT(*) DESC) AS rk
7     FROM tamal_1.ocurre_en
8     GROUP BY street
9 ),
10 calles_top AS (
11     SELECT street
12     FROM ranking_calles
13     WHERE rk <= 10
14 ),
15 ofensas_calles AS (
16     SELECT
17         o.street,
18         f.offense_description,
19         COUNT(*) AS freq,
20         ROW_NUMBER() OVER (
21             PARTITION BY o.street
22             ORDER BY COUNT(*) DESC) AS rn
23     FROM tamal_1.ocurre_en o
24     JOIN tamal_1.comete c USING (incident_number)
```

```

25 JOIN tamal_1.ofensa f USING (offense_code)
26 WHERE o.street IN (SELECT street FROM calles_top)
27 GROUP BY o.street, f.offense_description
28 )
29 SELECT
30     street,
31     offense_description AS delito_mas_comun,
32     freq
33 FROM ofensas_calles
34 WHERE rn = 1
35 ORDER BY freq DESC;

```

Consulta 6: Tipo de crimen y su distritos más comunes.

Código 20: Consulta 6.

```

1 WITH ranking_distritos AS (
2     SELECT
3         f.offense_description,
4         o.codigo AS distrito,
5         COUNT(*) AS total_incidentes,
6         ROW_NUMBER() OVER (
7             PARTITION BY f.offense_description
8             ORDER BY COUNT(*) DESC) AS rn
9     FROM tamal_1.comete c
10    JOIN tamal_1.ofensa f USING (offense_code)
11    JOIN tamal_1.ocurre_en o USING (incident_number)
12    GROUP BY f.offense_description, o.codigo
13 )
14 SELECT
15     offense_description,
16     distrito,
17     total_incidentes
18 FROM ranking_distritos
19 WHERE rn <= 10
20 ORDER BY offense_description, total_incidentes DESC;

```

3.4. Optimización

Índices para mejorar JOIN y filtros por claves foráneas.

Código 21: Optimización con índices de JOIN y filtros.

```

1 CREATE INDEX idx_comete_offense_code ON Tamal_1.Comete(OFFENSE_CODE);
2 CREATE INDEX idx_comete_incident_number ON
   ↪ Tamal_1.Comete(INCIDENT_NUMBER);

```

```
3 CREATE INDEX idx_ocurre_incident_number ON  
    ↪ Tamal_1.Ocurre_en(INCIDENT_NUMBER);  
4 CREATE INDEX idx_ocurre_codigo ON Tamal_1.Ocurre_en(Codigo);  
5 CREATE INDEX idx_ocurre_street ON Tamal_1.Ocurre_en(STREET);  
6 CREATE INDEX idx_incidente_date ON Tamal_1.Incidente(OCURRED_ON_DATE);
```

Índices para filtros por año, mes, día y hora.

Código 22: Optimización de índices de filtros.

```
1 CREATE INDEX idx_comete_year ON Tamal_1.Comete(YEAR);  
2 CREATE INDEX idx_comete_month ON Tamal_1.Comete(MONTH);  
3 CREATE INDEX idx_comete_hour ON Tamal_1.Comete(HOUR);  
4 CREATE INDEX idx_comete_day_of_week ON Tamal_1.Comete(DAY_OF_WEEK);
```

4. Hito 4

4.1. Tiempos de Ejecución de Consultas No Optimizadas

- Consulta 1:

```

QUERY PLAN
-----
Finalize GroupAggregate (cost=18098.41..18098.46 rows=12 width=35) (actual time=1809.988..1818.328 rows=12 loops=1)
  Group Key: c.config
  -- Gather Merge (cost=10034.41..18039.79 rows=12 width=35) (actual time=1809.802..1818.380 rows=12 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    --> Sort (cost=17034.40..17034.43 rows=12 width=35) (actual time=1798.066..1798.483 rows=12 loops=1)
      Sort Key: c.config
      Sort Method: quicksort Memory: 256kB
      Worker 0: Sort Method: quicksort Memory: 256kB
      --> Partial HashAggregate (cost=17034.86..17034.18 rows=12 width=35) (actual time=1798.398..1798.416 rows=12 loops=2)
        Group Key: c.config
        Batches: 1 Memory Usage: 264kB
        Worker 0: Batches: 1 Memory Usage: 264kB
        --> Parallel Hash Join (cost=8221.75..13893.75 rows=173135 width=5) (actual time=1854.841..1867.316 rows=148856 loops=2)
          Hash Cond: ((c.offense_number)::text = (c.incident_number)::text)
          --> Parallel Seq Scan on comets c (cost=0.86..3941.25 rows=173125 width=13) (actual time=0.611..268.870 rows=148856 loops=2)
          --> Parallel Hash (cost=1708.68..3028.68 rows=153368 width=14) (actual time=0.881..488.268 rows=132663 loops=1)
            Buckets: 262144 Batches: 4 Memory Usage: 516kB
            --> Parallel Seq Scan on security s (cost=0.46..3528.68 rows=153368 width=14) (actual time=0.622..246.131 rows=132663 loops=1)
Planning Time: 8.484 ms
Execution Time: 1818.409 ms
(23 rows)

```

Figura 4: Caption

- Consulta 2:

```

QUERY PLAN
-----
WindowAgg (cost=7983.88..7983.15 rows=1512 width=82) (actual time=1865.259..1865.933 rows=12 loops=1)
  Run Condition: (rank() OVER () <= 3)
  --> Sort (cost=7983.88..7987.68 rows=1512 width=80) (actual time=1863.242..1865.968 rows=1280 loops=1)
    Sort Key: c.day_of_week, count(*) DESC
    Sort Method: quicksort Memory: 1584kB
    --> Finalize HashAggregate (cost=4986.91..4928.43 rows=1512 width=80) (actual time=1861.946..1862.988 rows=1280 loops=1)
      Group Key: c.day_of_week, c.offense_description
      Batches: 1 Memory Usage: 321kB
      --> Gather (cost=1731.25..4897.37 rows=1512 width=80) (actual time=1851.514..1858.886 rows=2822 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        --> Partial HashAggregate (cost=3731.25..5748.37 rows=1512 width=80) (actual time=1868.489..1861.618 rows=1211 loops=2)
          Group Key: c.day_of_week, c.offense_description
          Batches: 1 Memory Usage: 321kB
          Worker 0: Batches: 1 Memory Usage: 321kB
          --> Hash Join (cost=7.88..4037.82 rows=179125 width=36) (actual time=0.724..678.637 rows=148856 loops=2)
            Hash Cond: ((c.offense_code)::text = (c2.offense_code)::text)
            --> Parallel Seq Scan on comets c (cost=0.49..3941.25 rows=173125 width=12) (actual time=0.618..164.502 rows=148856 loops=2)
            --> Hash (cost=17.517 rows=217 width=82) (actual time=0.673..8.875 rows=217 loops=2)
              Buckets: 1824 Batches: 1 Memory Usage: 224kB
              --> Seq Scan on offenses o2 (cost=0.49..5.17 rows=217 width=82) (actual time=0.834..6.322 rows=217 loops=2)
Planning Time: 8.955 ms
Execution Time: 1867.210 ms
(23 rows)

```

Figura 5: Caption

- Consulta 3:

```

QUERY PLAN
-----
Sort (cost=7198.77..7198.82 rows=21 width=36) (actual time=618.239..618.448 rows=217 loops=1)
  Sort Key: o.offense_description
  Sort Method: quicksort Memory: 488kB
  --> Hash Join (cost=7068.53..7198.38 rows=21 width=36) (actual time=612.415..618.834 rows=217 loops=1)
    Hash Cond: ((com.offense_code)::text = (o.offense_code)::text)
    --> Subquery Scan on com (cost=982.31..7198.37 rows=21 width=8) (actual time=611.765..617.854 rows=217 loops=1)
      Filter: (com.nv = 3)
      --> WindowAgg (cost=7852.31..7137.27 rows=4248 width=22) (actual time=611.763..618.828 rows=217 loops=1)
        Run Condition: (row_number() OVER () <= 3)
        --> Sort (cost=7852.31..7862.93 rows=4248 width=10) (actual time=611.747..613.778 rows=3889 loops=1)
          Sort Key: comets.offense_code, count(*) DESC
          Sort Method: quicksort Memory: 3976kB
          --> Finalize HashAggregate (cost=4783.43..4796.31 rows=4248 width=14) (actual time=602.169..605.216 rows=3889 loops=1)
            Group Key: comets.offense_code, comets.hour
            Batches: 1 Memory Usage: 321kB
            --> Gather (cost=4268.49..4723.97 rows=4248 width=14) (actual time=673.582..599.783 rows=7104 loops=1)
              Workers Planned: 1
              Workers Launched: 1
              --> Partial HashAggregate (cost=3204.69..4329.17 rows=4248 width=14) (actual time=365.482..570.376 rows=3572 loops=1)
                Group Key: comets.offense_code, comets.hour
                Batches: 1 Memory Usage: 321kB
                Worker 0: Batches: 1 Memory Usage: 321kB
                --> Parallel Seq Scan on comets c (cost=0.86..3941.25 rows=173125 width=6) (actual time=0.613..228.898 rows=148856 loops=2)
                  --> Hash (cost=5.17..5.17 rows=217 width=22) (actual time=0.632..8.634 rows=217 loops=1)
                    Buckets: 3024 Batches: 1 Memory Usage: 224kB
                    --> Seq Scan on offenses o (cost=0.48..5.17 rows=217 width=22) (actual time=0.619..8.384 rows=217 loops=1)
Planning Time: 9.463 ms
Execution Time: 619.186 ms
(28 rows)

```

Figura 6: Caption

- Consulta 4:


```

QUERY PLAN

Subquery Scan on con_diferencia (cost=9767.87..9067.78 rows=1321 width=50) (actual time=2178.176..2095.138 rows=488 loops=1)
  -> WindowAgg (cost=9767.87..9013.43 rows=1321 width=50) (actual time=2178.172..2081.812 rows=488 loops=1)
    -> Finalize GroupAggregate (cost=9767.87..9095.18 rows=1321 width=19) (actual time=2177.391..2092.573 rows=488 loops=1)
      Group Key: c.callep, (date_trunc('month', text, 1, occurred_on_date))
      -> Gather Merge (cost=9767.87..9072.78 rows=777 width=19) (actual time=2176.675..2081.133 rows=968 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        -> Partial GroupAggregate (cost=9767.86..8785.34 rows=777 width=19) (actual time=1886.994..2287.948 rows=888 loops=2)
          Group Key: o.codigo, (date_trunc('month', text, 1, occurred_on_date))
          -> Sort (cost=9767.86..7358.88 rows=777 width=13) (actual time=1866.668..2169.818 rows=12863 loops=2)
            Sort Key: o.codigo, (date_trunc('month', text, 1, occurred_on_date))
            Sort Method: external merge  Disk: 3076kB
            -> Parallel Hash Join (cost=8808.88..8728.55 rows=777 width=11) (actual time=1122.352..1545.518 rows=12863 loops=2)
              Hash Cond: ((c.incident_number)::text = (i.incident_number)::text)
              -> Parallel Seq Scan on ocurre_en o (cost=0.00..3528.68 rows=153368 width=14) (actual time=0.889..178.162 rows=1328
40 loops=2)
              -> Parallel Hash (cost=9798.36..4790.36 rows=777 width=19) (actual time=718.369..718.372 rows=12863 loops=2)
                Buckets: 131872 (originally 380) Batches: 4 (originally 13) Memory Usage: 4589kB
                -> Parallel Seq Scan on incidente i (cost=0.00..4790.36 rows=777 width=19) (actual time=0.812..139.668 rows=1
12863 loops=2)
Filter: ((EXTRACT(year FROM occurred_on_date) = '2015')::numeric) AND (EXTRACT(year FROM occurred_on_date)
Planning Time: 8.716 ms
Execution Time: 2096.408 ms
(22 rows)

```

Figura 7: Caption

- Consulta 5:

```

QUERY PLAN

Sort (cost=98878.53..98882.25 rows=1489 width=48) (actual time=2837.755..2837.898 rows=18 loops=1)
  Sort Key: ofensas_calles.freq DESC
  Sort Method: quicksort  Memory: 25kB
  -> Subquery Scan on ofensas_calles (cost=62574.05..98880.06 rows=1489 width=48) (actual time=2738.397..2837.856 rows=18 loops=1)
    Filter: (ofensas_calles.fn = 1)
    -> WindowAgg (cost=62574.05..94278.65 rows=297713 width=56) (actual time=2738.394..2837.833 rows=18 loops=1)
      Run Condition: (row_number() OVER (?) <= 1)
      -> Incremental Sort (cost=62574.05..89868.67 rows=297713 width=48) (actual time=2738.388..2836.658 rows=1465 loops=1)
        Sort Key: o.street, (count(*) DESC)
        Presorted Key: o.street
        Full-sort Groups: 10 Sort Method: quicksort  Average Memory: 30kB Peak Memory: 30kB
        Pre-sorted Groups: 10 Sort Method: quicksort  Average Memory: 39kB Peak Memory: 39kB
        -> GroupAggregate (cost=62565.91..68526.17 rows=297713 width=48) (actual time=2724.724..2833.726 rows=1465 loops=1)
          Group Key: o.street, f.offense_description
          -> Sort (cost=62565.91..63310.19 rows=297713 width=40) (actual time=2724.786..2774.672 rows=58187 loops=1)
            Sort Key: o.street, f.offense_description
            Sort Method: external merge  Disk: 2776kB
            -> Hash Join (cost=15373.40..27357.49 rows=297713 width=40) (actual time=1806.293..2601.533 rows=58187 loops=1)
              Hash Cond: (o.street = ranking_calles.street)
              -> Hash Join (cost=9465.72..20667.12 rows=297713 width=40) (actual time=493.919..1823.126 rows=297713 loops=1)
                Hash Cond: ((c.offense_code)::text = (f.offense_code)::text)
                -> Hash Join (cost=9457.84..19862.47 rows=297713 width=16) (actual time=493.467..1380.838 rows=297713 loops=1)
                  Hash Cond: ((c.incident_number)::text = (o.incident_number)::text)
                  -> Seq Scan on comete c (cost=0.00..5167.13 rows=297713 width=15) (actual time=0.086..235.636 rows=2977
13 loops=1)
                  -> Hash (cost=4688.26..4688.26 rows=264126 width=23) (actual time=492.771..492.774 rows=264126 loops=1)
                    Buckets: 131872 Batches: 4 Memory Usage: 4589kB
                    -> Seq Scan on ocurre_en o (cost=0.00..4688.26 rows=264126 width=23) (actual time=0.088..221.647
rows=264126 loops=1)
                    -> Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.424..0.426 rows=217 loops=1)
                      Buckets: 1024 Batches: 1 Memory Usage: 22kB

```

```

-> Seq Scan on ofensa f (cost=0.00..5.17 rows=217 width=32) (actual time=0.019..0.201 rows=217 loops=1)
-> Hash (cost=5873.34..5873.34 rows=2747 width=12) (actual time=512.329..512.445 rows=10 loops=1)
  Buckets: 4096 Batches: 1 Memory Usage: 33kB
  -> Subquery Scan on ranking_calles (cost=5797.80..5873.34 rows=2747 width=12) (actual time=512.276..512.428 r
ows=10 loops=1)
    -> WindowAgg (cost=5797.80..5845.87 rows=2747 width=28) (actual time=512.273..512.412 rows=10 loops=1)
      Run Condition: (rank() OVER (?) <= 10)
      -> Sort (cost=5797.80..5868.67 rows=2747 width=20) (actual time=512.260..512.377 rows=12 loops=1)
        Sort Key: (count(*) DESC)
        Sort Method: quicksort  Memory: 343kB
        -> Finalize HashAggregate (cost=5613.43..5640.90 rows=2747 width=20) (actual time=585.695..
588.853 rows=3942 loops=1)
          Group Key: ocurre_en.street
          Batches: 1 Memory Usage: 721kB
          -> Gather (cost=5297.52..5599.69 rows=2747 width=20) (actual time=477.220..494.824 ro
ws=7222 loops=1)
            Workers Planned: 1
            Workers Launched: 1
            -> Partial HashAggregate (cost=4297.52..4324.99 rows=2747 width=20) (actual tim
e=472.324..475.623 rows=3611 loops=2)
              Group Key: ocurre_en.street
              Batches: 1 Memory Usage: 721kB
              Worker 0: Batches: 1 Memory Usage: 721kB
              -> Parallel Seq Scan on ocurre_en (cost=0.00..3528.68 rows=153368 width=1
2) (actual time=0.011..282.634 rows=132063 loops=2)
Planning Time: 1.572 ms
Execution Time: 2838.929 ms
(51 rows)

```

Figura 8: Comparación de resultados antes (arriba) y después (abajo) del uso de índices.

- Consulta 6:

```

Subquery Scan on ranking_distributos (cost=16858.41..16592.39 rows=2592 width=39) (actual time=2268.788..2278.979 rows=1759 loops=1)
  -> WindowAgg (cost=16858.41..16568.47 rows=2592 width=39) (actual time=2268.776..2270.139 rows=1759 loops=1)
    Row Condition (row_number() OVER (?) <= 18)
    -> Incremental Sort (cost=16858.41..16521.11 rows=2592 width=39) (actual time=2268.762..2272.112 rows=2865 loops=1)
      Sort Key: f.offense_description, (count(*) DESC)
      Presorted Key: f.offense_description
      Full-sort Groups: 57 Sort Method: quicksort Average Memory: 2948 Peak Memory: 2948
      -> Finalize GroupAggregate (cost=16858.41..16568.47 rows=2592 width=39) (actual time=2268.762..2269.864 rows=2865 loops=1)
        Group Key: f.offense_description, o.codigo
        -> Gather Merge (cost=16858.41..16526.75 rows=2592 width=39) (actual time=2268.474..2261.214 rows=3869 loops=1)
          Workers Planned: 1
          Workers Launched: 1
          -> Sort (cost=1685.48..16861.12 rows=2592 width=39) (actual time=2231.769..2232.973 rows=1924 loops=2)
            Sort Key: f.offense_description, o.codigo
            Sort Method: quicksort Memory: 48608
            Worker 0: Sort Method: quicksort Memory: 28468
            -> Partial HashAggregate (cost=16858.74..16869.97 rows=2592 width=39) (actual time=2223.187..2226.881 rows=1924 loops=2)
              Group Key: f.offense_description, o.codigo
              Batches: 1 Memory Usage: 36948
              Worker 0: Batches: 1 Memory Usage: 36948
              -> Hash Join (cost=629.48..11578.11 rows=175125 width=31) (actual time=1681.582..1689.719 rows=148856 loops=2)
                Hash Cond: ((c.offense_code) text = (f.offense_code) text)
                -> Parallel Hash Join (cost=622.71..13893.75 rows=175125 width=7) (actual time=1618.914..1387.237 rows=148856 loops=2)
                  Hash Cond: ((c.incident_number) text = (f.incident_number) text)
                  -> Parallel Seq Scan on conete c (cost=0.48..3941.25 rows=175125 width=15) (actual time=0.812..252.776 rows=148856 loops=2)
                    -> Parallel Hash (cost=9328.61..3528.48 rows=153368 width=14) (actual time=495.540..405.543 rows=112863 loops=2)
                      Buckets: 262184 Batches: 4 Memory Usage: 51868
                      -> Parallel Seq Scan on secure_sx s (cost=0.48..3528.48 rows=153368 width=14) (actual time=0.815..289.373 rows=112863 loops=2)
                        -> Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.637..0.688 rows=217 loops=2)
                          Buckets: 1824 Batches: 1 Memory Usage: 2268
                          -> Seq Scan on ofensa f (cost=0.48..5.17 rows=217 width=32) (actual time=0.628..0.299 rows=217 loops=2)
Planning Time: 8.999 ms
Execution Time: 2279.787 ms
(21 rows)

```

Figura 9: Caption

4.2. Tiempos de Ejecución de Consultas Optimizadas

- Consulta 1:

```

QUERY PLAN

Finalize GroupAggregate (cost=18834.41..18836.86 rows=12 width=35) (actual time=1773.621..1779.991 rows=12 loops=1)
  Group Key: o.codigo
  -> Gather Merge (cost=18834.41..18835.79 rows=12 width=35) (actual time=1773.685..1779.942 rows=24 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    -> Sort (cost=17934.08..17934.43 rows=12 width=35) (actual time=1762.357..1762.374 rows=12 loops=2)
      Sort Key: o.codigo
      Sort Method: quicksort Memory: 2568
      -> Partial HashAggregate (cost=17934.86..17934.18 rows=12 width=35) (actual time=1762.289..1762.387 rows=12 loops=2)
        Group Key: o.codigo
        Batches: 1 Memory Usage: 2468
        Worker 0: Batches: 1 Memory Usage: 2468
        -> Parallel Hash Join (cost=18211.75..13893.75 rows=175125 width=5) (actual time=1868.258..1453.669 rows=148856 loops=2)
          Hash Cond: ((c.incident_number) text = (f.incident_number) text)
          -> Parallel Seq Scan on conete c (cost=0.48..3941.25 rows=175125 width=15) (actual time=0.811..243.328 rows=148856 loops=2)
            -> Parallel Hash Join (cost=18211.75..13893.75 rows=175125 width=11) (actual time=487.242..489.243 rows=112863 loops=2)
              Hash Cond: ((c.offense_code) text = (f.offense_code) text)
              -> Parallel Seq Scan on secure_sx s (cost=0.48..3528.48 rows=153368 width=14) (actual time=0.816..227.834 rows=112863 loops=2)
                -> Parallel Hash (cost=9328.61..3528.48 rows=153368 width=14) (actual time=495.540..405.543 rows=112863 loops=2)
                  Buckets: 262184 Batches: 4 Memory Usage: 51868
                  -> Parallel Seq Scan on secure_sx s (cost=0.48..3528.48 rows=153368 width=14) (actual time=0.816..227.834 rows=112863 loops=2)
Planning Time: 2.338 ms
Execution Time: 1780.129 ms
(21 rows)

```

Figura 10: Caption

- Consulta 2:

```

QUERY PLAN

WindowAgg (cost=7883.88..7834.12 rows=1512 width=52) (actual time=1868.699..1862.629 rows=22 loops=1)
  Row Condition: (week_o2 <= 4)
  -> Sort (cost=7883.88..7867.66 rows=1512 width=54) (actual time=1868.683..1861.529 rows=1288 loops=1)
    Sort Key: o.day_of_week, o2.offense_description
    Sort Method: quicksort Memory: 15584
    -> Finalize HashAggregate (cost=7883.81..8024.43 rows=1512 width=40) (actual time=1857.245..1858.482 rows=1288 loops=1)
      Group Key: o.day_of_week, o2.offense_description
      Batches: 1 Memory Usage: 32168
      -> Gather (cost=6731.25..6897.57 rows=1512 width=44) (actual time=1844.631..1854.151 rows=2434 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        -> Partial HashAggregate (cost=6731.25..6766.37 rows=1512 width=40) (actual time=1835.848..1836.374 rows=1217 loops=2)
          Group Key: o.day_of_week, o2.offense_description
          Batches: 1 Memory Usage: 32168
          Worker 0: Batches: 1 Memory Usage: 32168
          -> Hash Join (cost=67.88..4817.82 rows=75125 width=36) (actual time=0.726..688.639 rows=148856 loops=2)
            Hash Cond: ((c.offense_code) text = (o2.offense_code) text)
            -> Parallel Seq Scan on conete c (cost=0.48..3941.25 rows=175125 width=15) (actual time=0.816..189.716 rows=148856 loops=2)
              -> Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.682..0.685 rows=217 loops=2)
                Buckets: 1824 Batches: 1 Memory Usage: 2268
                -> Seq Scan on ofensa o2 (cost=0.48..5.17 rows=217 width=32) (actual time=0.626..0.338 rows=217 loops=2)
Planning Time: 1.281 ms
Execution Time: 1862.789 ms
(21 rows)

```

Figura 11: Caption

- Consulta 3:

```

QUERY PLAN

Sort (cost=7198.77..7198.82 rows=21 width=38) (actual time=627.838..628.848 rows=217 loops=1)
  Sort Key: f.offense_description
  Sort Method: quicksort Memory: 48608
  -> Hash Join (cost=7188.00..6216.38 rows=21 width=38) (actual time=626.968..627.583 rows=217 loops=1)
    Hash Cond: ((c.offense_code) text = (f.offense_code) text)
    -> Seq Scan on ofensa f (cost=7188.00..6216.38 rows=21 width=38) (actual time=626.971..626.986 rows=217 loops=1)
      Filter: (code <= 1)
      -> WindowAgg (cost=7188.21..7197.27 rows=4248 width=22) (actual time=626.278..626.968 rows=217 loops=1)
        Row Condition: (row_number() OVER (?) <= 1)
        -> Sort (cost=625.13..762.49 rows=4248 width=14) (actual time=626.356..627.798 rows=3849 loops=1)
          Sort Key: conete.offense_code, (count(*) DESC)
          Sort Method: quicksort Memory: 39548
          -> Finalize HashAggregate (cost=6793.83..6796.31 rows=4248 width=14) (actual time=618.585..613.719 rows=3849 loops=1)
            Group Key: conete.offense_code, conete.hour
            Batches: 1 Memory Usage: 72168
            Worker 0: Batches: 1 Memory Usage: 72168
            -> Gather (cost=6254.69..6271.97 rows=4248 width=14) (actual time=581.488..599.426 rows=7741 loops=1)
              Workers Planned: 1
              Workers Launched: 1
              -> Partial HashAggregate (cost=6254.69..6269.17 rows=4248 width=14) (actual time=573.559..576.885 rows=3578 loops=2)
                Group Key: conete.offense_code, conete.hour
                Batches: 1 Memory Usage: 72168
                Worker 0: Batches: 1 Memory Usage: 72168
                -> Parallel Seq Scan on conete c (cost=0.48..3941.25 rows=175125 width=15) (actual time=0.813..218.483 rows=148856 loops=2)
                  -> Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.588..0.587 rows=217 loops=1)
                    Buckets: 1824 Batches: 1 Memory Usage: 2268
                    -> Seq Scan on ofensa o (cost=0.48..5.17 rows=217 width=32) (actual time=0.618..0.278 rows=217 loops=1)
Planning Time: 0.571 ms
Execution Time: 628.342 ms
(21 rows)

```

Figura 12: Caption

- Consulta 4:

```

QUERY PLAN
-----
Subquery Scan on con.diferencia (cost=9767.87..9967.76 rows=1321 width=59) (actual time=2172.119..2496.463 rows=880 loops=1)
  -> WindowAgg (cost=9767.87..9921.43 rows=1321 width=27) (actual time=2172.115..2492.215 rows=880 loops=1)
    -> Finalize GroupAggregate (cost=5767.87..5885.18 rows=1321 width=10) (actual time=2171.576..2492.896 rows=880 loops=1)
      Group Key: o.codigo, (data_trunc('month'::text, i.occurred_on_date))
      -> Gather Merge (cost=9767.87..9877.76 rows=177 width=59) (actual time=2170.636..2492.415 rows=968 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        -> Partial GroupAggregate (cost=8767.86..8785.34 rows=177 width=10) (actual time=1970.876..2279.388 rows=880 loops=2)
          Group Key: o.codigo, (data_trunc('month'::text, i.occurred_on_date))
          -> Sort (cost=8767.86..8785.34 rows=177 width=11) (actual time=1974.565..2237.583 rows=12863 loops=2)
            Sort Method: external merge  Memory: 343kB
            Worker 0: Sort Method: external merge  Disk: 3326kB
            -> Parallel Hash Scan on ocurre_en o (cost=87.86..3328.48 rows=13388 width=14) (actual time=1227.837..1534.288 rows=13263 loops=2)
              Hash Cond: ((i.incident_number)::text = (i.incident_number)::text)
              -> Parallel Seq Scan on ocurre_en o (cost=87.86..3328.48 rows=13388 width=14) (actual time=8.807..176.127 rows=1328
83 loops=2)
              -> Parallel Hash (cost=8780.36..4798.36 rows=177 width=10) (actual time=788.669..788.681 rows=12863 loops=2)
                Buckets: 131872 (originally 2688)  Batches: 4 (originally 13)  Memory Usage: 4736kB
                -> Parallel Seq Scan on incidente i (cost=88..4798.36 rows=177 width=10) (actual time=0.812..366.222 rows=1
12863 loops=2)
                Filter: ((EXTRACT(year FROM occurred_on_date) = '2015'::numeric) AND (EXTRACT(year FROM occurred_on_date)
= '2015'::numeric))
Planning Time: 9.818 ms
Execution Time: 2497.427 ms
(22 rows)

```

Figura 13: Caption

- Consulta 5:

```

QUERY PLAN
-----
Sort (cost=98813.28..98817.00 rows=1489 width=48) (actual time=2867.925..2868.132 rows=10 loops=1)
  Sort Key: ofensas_calles.freq DESC
  Sort Method: quicksort  Memory: 25kB
  -> Subquery Scan on ofensas_calles (cost=62508.80..97934.81 rows=1489 width=48) (actual time=2765.059..2868.098 rows=10 loops=1)
    Filter: (ofensas_calles.rn = 1)
    -> WindowAgg (cost=62508.80..94213.40 rows=297713 width=56) (actual time=2765.855..2868.074 rows=10 loops=1)
      Run Condition: (row_number()) OVER (?) <= 1
      -> Incremental Sort (cost=62508.80..89803.42 rows=297713 width=48) (actual time=2765.038..2866.856 rows=1465 loops=1)
        Sort Key: o.street, (count(*)) DESC
        Presorted Key: o.street
        Full-sort Groups: 10  Sort Method: quicksort  Average Memory: 38kB  Peak Memory: 38kB
        Pre-sorted Groups: 10  Sort Method: quicksort  Average Memory: 39kB  Peak Memory: 39kB
        -> GroupAggregate (cost=62508.66..68454.92 rows=297713 width=48) (actual time=2758.997..2863.798 rows=1465 loops=1)
          Group Key: o.street, f.offense_description
          -> Sort (cost=62508.66..63244.94 rows=297713 width=48) (actual time=2758.977..2803.489 rows=58187 loops=1)
            Sort Key: o.street, f.offense_description
            Sort Method: external merge  Disk: 2776kB
            -> Hash Join (cost=15308.15..27292.24 rows=297713 width=40) (actual time=978.490..2628.519 rows=58187 loops=1)
              Hash Cond: (o.street = ranking_calles.street)
              -> Hash Join (cost=9465.72..28667.12 rows=297713 width=40) (actual time=585.402..1882.678 rows=297713 loops=1)
                Hash Cond: ((c.offense_code)::text = (f.offense_code)::text)
                -> Hash Join (cost=9457.84..19862.47 rows=297713 width=16) (actual time=504.961..1428.728 rows=297713 loops=1)
                  Hash Cond: ((c.incident_number)::text = (o.incident_number)::text)
                  -> Seq Scan on comete c (cost=0.00..5167.13 rows=297713 width=15) (actual time=0.006..251.226 rows=2977
13 loops=1)
                  -> Hash (cost=4688.26..4688.26 rows=264126 width=23) (actual time=504.690..504.694 rows=264126 loops=1)
                    Buckets: 131872  Batches: 4  Memory Usage: 4589kB
                    -> Seq Scan on ocurre_en o (cost=0.00..4688.26 rows=264126 width=23) (actual time=0.007..231.518
rows=264126 loops=1)
                    -> Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.428..0.431 rows=217 loops=1)
                      Buckets: 1024  Batches: 1  Memory Usage: 22kB

```

```

-> Seq Scan on ofensa f (cost=0.00..5.17 rows=217 width=32) (actual time=0.015..0.211 rows=217 loops=1)
-> Hash (cost=5888.09..5888.09 rows=2747 width=12) (actual time=473.076..473.252 rows=10 loops=1)
  Buckets: 4096  Batches: 1  Memory Usage: 33kB
  -> Subquery Scan on ranking_calles (cost=5732.55..5888.09 rows=2747 width=12) (actual time=473.007..473.224 r
ows=10 loops=1)
    -> WindowAgg (cost=5732.55..5780.62 rows=2747 width=28) (actual time=473.005..473.286 rows=10 loops=1)
      Run Condition: (rank() OVER (?) <= 10)
      -> Sort (cost=5732.55..5739.42 rows=2747 width=20) (actual time=472.992..473.169 rows=12 loops=1)
        Sort Key: count(*) DESC
        Sort Method: quicksort  Memory: 343kB
        -> Finalize HashAggregate (cost=5548.18..5575.65 rows=2747 width=20) (actual time=463.177..
469.377 rows=3942 loops=1)
          Group Key: ocurre_en.street
          Buckets: 1  Memory Usage: 721kB
          -> Gather (cost=1800.42..5520.71 rows=5494 width=20) (actual time=1.551..458.206 rows
=3993 loops=1)
            Workers Planned: 2
            Workers Launched: 2
            -> Partial GroupAggregate (cost=0.42..3971.31 rows=2747 width=20) (actual time=
0.556..445.542 rows=1331 loops=3)
              Group Key: ocurre_en.street
              -> Parallel Index Only Scan using idx_ocurre_street on ocurre_en (cost=0.
42..3393.58 rows=110052 width=12) (actual time=0.081..237.972 rows=88042 loops=3)
                Heap Fetches: 0
Planning Time: 1.148 ms
Execution Time: 2868.944 ms
(50 rows)

```

Figura 14: Comparación de resultados antes (arriba) y después (abajo) del uso de índices.

- Consulta 6:

```

Subquery Scan on ranking_district (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2319.277..2360.228 rows=1759 loops=1)
  -> Memory Scan (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2319.273..2360.272 rows=1759 loops=1)
    Run Condition: (row_number() OVER (??) <= 18)
    -> Incremental Sort (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2319.267..2338.276 rows=2840 loops=1)
      Sort Key: f.offense_description, (count(*) DESC)
      Partitioned Key: f.offense_description
      Full-sort Groups: 57 Sort Method: quicksort Average Memory: 276K Peak Memory: 276K
      -> Finalize GroupAggregate (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2389.906..2333.983 rows=2840 loops=1)
        Group Key: f.offense_description, s.codigo
        -> Gather Merge (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2389.972..2328.256 rows=3848 loops=1)
          Workers Planned: 1
          Workers Launched: 1
          -> Sort (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2296.554..2297.749 rows=1924 loops=2)
            Sort Key: f.offense_description, s.codigo
            Sort Method: quicksort Memory: 280K
            Worker 0: Sort Method: quicksort Memory: 280K
            -> Partial HashAggregate (cost=16858.61..16592.38 rows=2592 width=39) (actual time=2287.527..2291.383 rows=1924 loops=2)
              Group Key: f.offense_description, s.codigo
              Buckets: 1 Memory Usage: 360K
              Worker 0: Buckets: 1 Memory Usage: 360K
              -> Hash Join (cost=1629.69..1370.31 rows=175125 width=22) (actual time=1847.187..1919.498 rows=18856 loops=2)
                Hash Cond: ((c.offense_code)::text = (f.offense_code)::text)
                -> Parallel Hash Scan (cost=822.70..1389.70 rows=175125 width=7) (actual time=1846.487..1469.291 rows=18856 loops=2)
                  Hash Cond: ((c.incident_number)::text = (f.incident_number)::text)
                  -> Parallel Seq Scan on crime c (cost=0.89..3981.23 rows=175125 width=15) (actual time=0.819..238.389 loops=2)
                    -> Parallel Hash (cost=3526.68..3526.68 rows=155368 width=14) (actual time=497.885..497.888 rows=132863 loops=2)
                      Buckets: 262140 Batches: 4 Memory Usage: 3306K
                      -> Parallel Seq Scan on source s s (cost=0.89..3526.68 rows=155368 width=14) (actual time=0.814..213.976 rows=132863 loops=2)
                        Hash (cost=5.17..5.17 rows=217 width=32) (actual time=0.668..8.678 rows=217 loops=2)
                        Buckets: 1824 Batches: 1 Memory Usage: 270K
                        -> Seq Scan on offense f (cost=0.89..5.17 rows=217 width=32) (actual time=0.631..8.322 rows=217 loops=2)
Planning Time: 1.973 ms
Execution Time: 2346.038 ms
(25 rows)

```

Figura 15: Caption

4.3. Comparación de Consultas: “Optimizadas v/s No Optimizadas”

Tabla 3: Caption

Consulta	Tiempo sin Índices [s]	Tiempo con Índices [s]	Tipo de Escaneo Antes	Tipo de Escaneo Después	Mejora [%]
Consulta 1					
Consulta 2					
Consulta 3					
Consulta 4					
Consulta 5					
Consulta 6					

4.4. Gráfico de consulta 1

Esta consulta presenta la evolución anual (2015–2018) de la cantidad total de crímenes registrados en cada distrito policial. Cada barra representa la suma de delitos por distrito y año.

Permite identificar distritos con tendencias persistentes de criminalidad o mejoras, ayudando a las autoridades a priorizar recursos policiales y tomar decisiones basadas en evidencia.

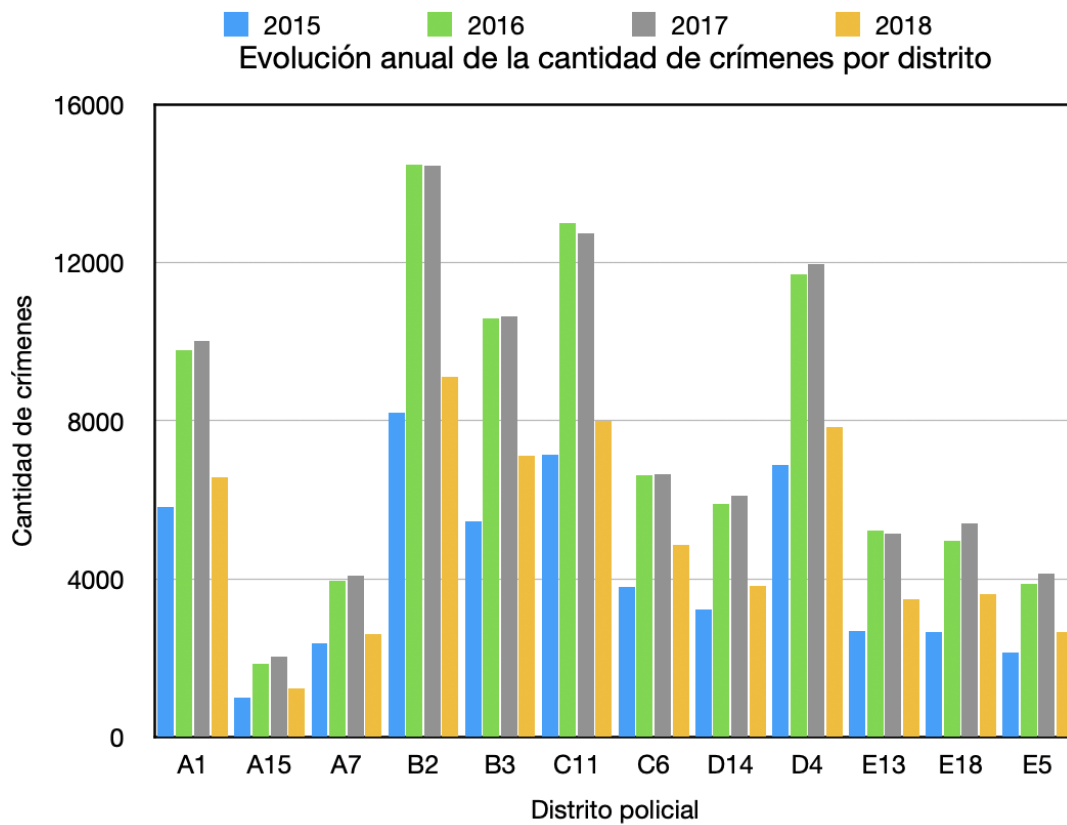


Figura 16: Consulta 1: Evolución anual de la cantidad de crímenes por distrito

4.5. Gráfico de consulta 2

Esta consulta muestra las calles que concentran la mayor cantidad de incidentes delictivos, ordenadas por frecuencia.

Identifica los puntos críticos dentro de la ciudad para focalizar patrullajes, instalación de cámaras o intervenciones comunitarias.

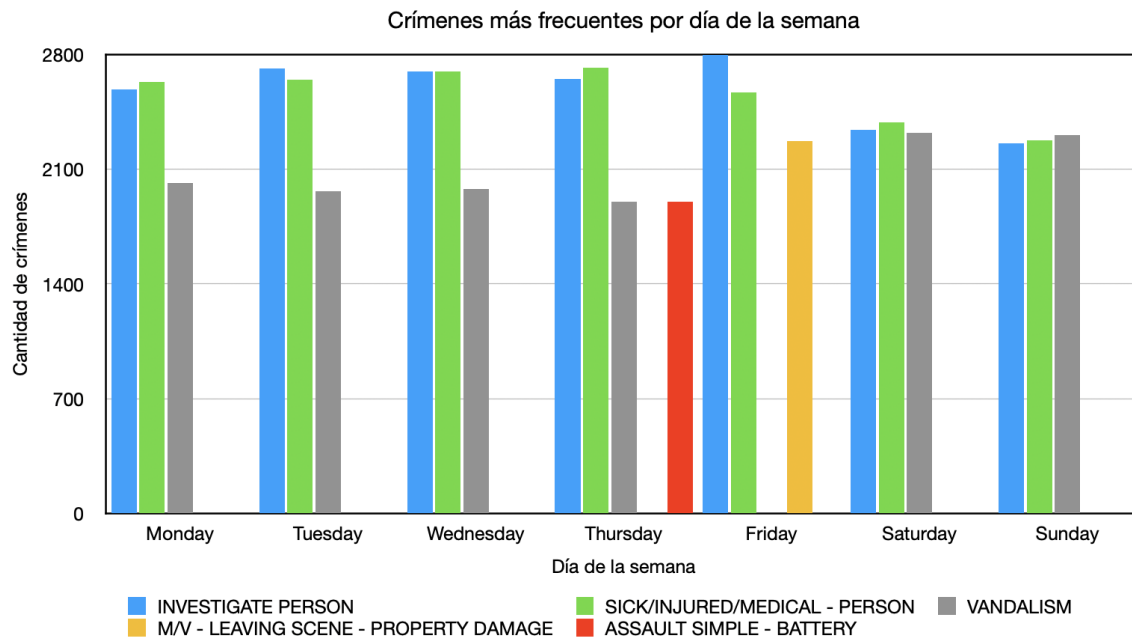


Figura 17: Consulta 2: Crímenes más frecuentes por día de la semana

4.6. Gráfico de consulta 5

Este gráfico representa los delitos más comunes cometidos cada día de la semana, con los cinco crímenes más frecuentes a lo largo de la semana.

Revela patrones temporales en el comportamiento delictivo, útil para planificar turnos policiales, campañas preventivas o alertas ciudadanas.

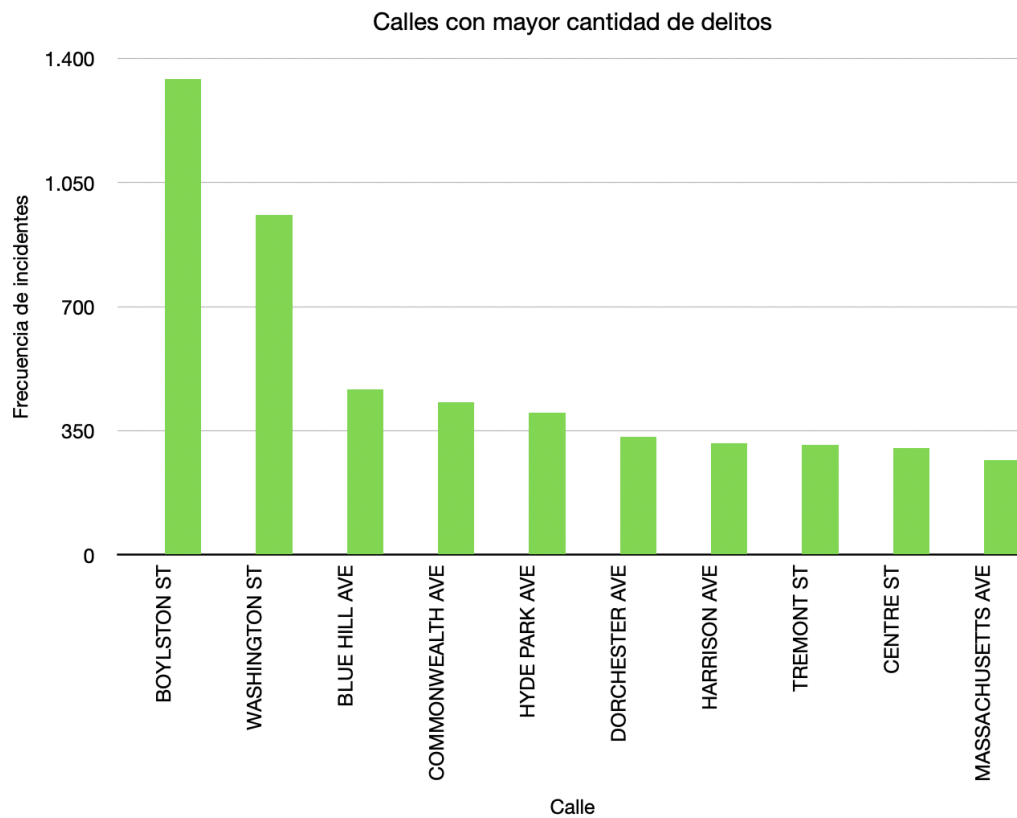


Figura 18: Consulta 5: Calles con mayor cantidad de delitos y sus tipos más comunes.